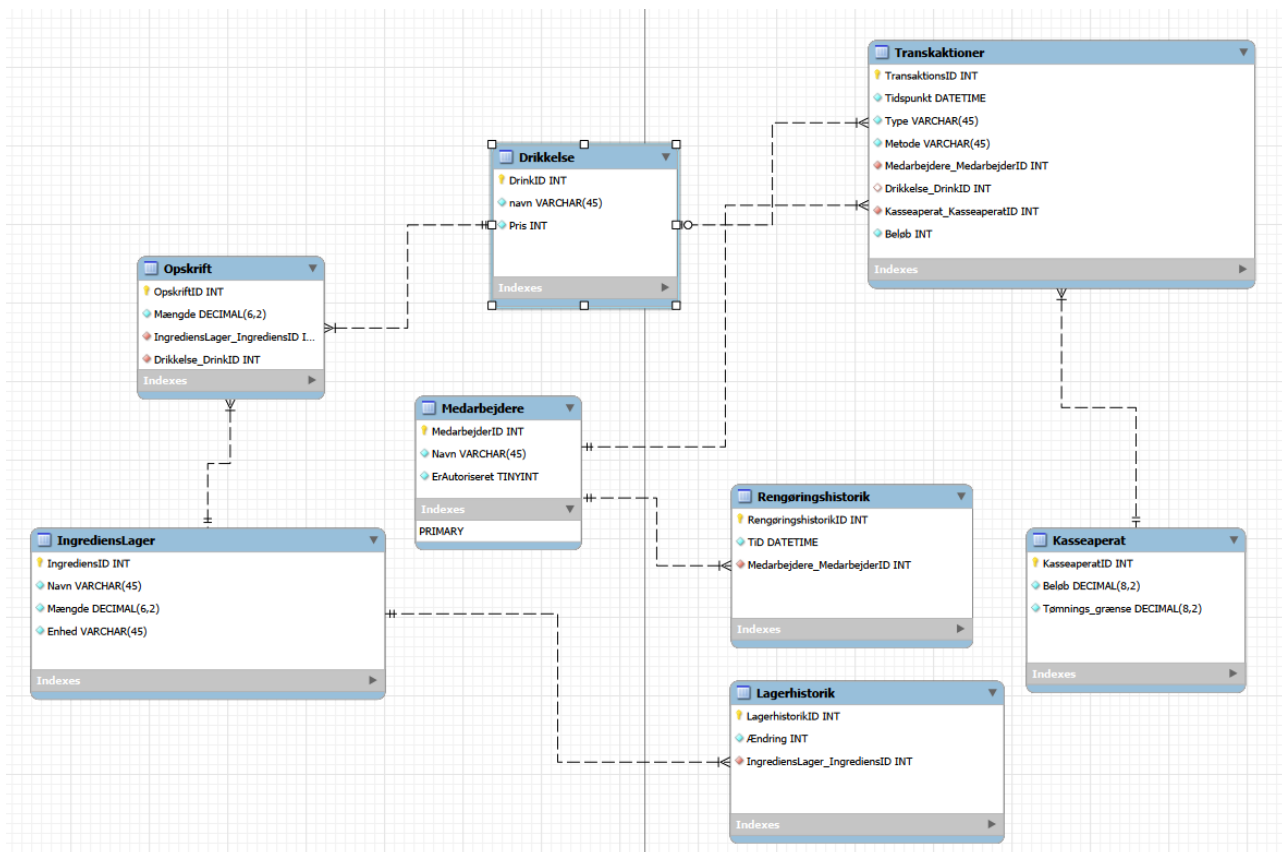


Mini Projekt



Diagrammet bruger 'NOT NULL' fordi hvis en af attributterne var NULL ville det påvirke processen og ødelægge systemet. For eksempel vil en Enhed der er NULL gøre det umuligt at vide om at vi har at gøre med 5000 kg eller gram kaffe. En undtagelse er Drikkelese_DrinkID som er nødt til at være NULL fordi foreign-key for kasseapparat-tabellen er forbundet til transaktioner. Så hvis den ikke kan være NULL så skal kassedamen lave et køb før kassen kan åbnes. Hvilket påvirker f.eks. tømningsgrænsen fordi kassen skal kunne åbnes op på et bestemt tidspunkt som nødvendigvis ikke er afhængig om at der lige er blevet lavet et køb. Kardinaliteten benytter sig af en til mange princippet, hvor f.eks. et kasseapparat kan lave mange transaktioner. Kasseapparatet sætter en foreign key i tabellen for transaktioner, og sådan er det for tabeller, der er mange i forholdet 1: N.

```

-- Trigger til Autorition
DELIMITER //
CREATE TRIGGER Autorisering
Before Insert ON Transaktioner
FOR EACH ROW
BEGIN
DECLARE autoriseret_status TINYINT;
SELECT ErAutoriseret INTO autoriseret_status
FROM Medarbejdere
WHERE MedarbejderID=NEW.Medarbejdere_MedarbejderID;
IF NEW.Type='Træk' AND autoriseret_status=0 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT='Medarbejder har ikke tilladelse';
END IF;
END //

```

Denne trigger laver en foregåar ved at før der kommer en tilførsel på transaktioner fra test script så kommer der hvad der kaldes for en trigger. Her sættes variabel "ErAutoriseret" fra tabellen medarbejdere ind i en ny variabel "autoriseret_status" fordi triggeren befinder sig i Transaktioner tabellen kan den ikke se imod medarbejder tabellen og derfor laves "autoriseret_status". MedarbejderID opdateres hver gang en ny medarbejder skal testes for autorition og hvis og typen er 'træk' og autoritions_status er 0 sender den teksten 'Medarbejder har ikke tilladelse'

```

DELIMITER //
CREATE TRIGGER OpdaterLager_og_kasse
After INSERT ON Transaktioner
FOR EACH ROW
BEGIN
IF NEW.Type='køb' AND NEW.Drikkelse_DrinkID IS NOT NULL THEN
UPDATE IngrediensLager IL
JOIN Opskrift O ON IL.IngrediensID=O.IngrediensLager_IngrediensID
SET IL.Mængde= IL.Mængde-O.Mængde
WHERE O.Drikkelse_DrinkID=NEW.Drikkelse_DrinkID;
INSERT INTO Lagerhistorik(Ændring, IngrediensLager_IngrediensID)
SELECT O.Mængde,O.IngrediensLager_IngrediensID
FROM Opskrift O
WHERE O.Drikkelse_DrinkID=NEW.Drikkelse_DrinkID;
END IF;
-- Opdatere kassen
IF NEW.Type='køb' AND NEW.metode='kontant' Then
UPDATE Kasseaperat
SET Beløb=Beløb+NEW.Beløb
WHERE KasseaperatID=NEW.Kasseaperat_KasseaperatID;
END IF;
IF NEW.Type='Træk' THEN
UPDATE Kasseaperat
SET Beløb=Beløb-NEW.Beløb
WHERE KasseaperatID=NEW.Kasseaperat_KasseaperatID;
END IF;
END//
DELIMITER ;

```

Der er blevet lavet anden trigger, der forekommer efter en insertion i transaktion tabellen. Dette er en lager opdatering som påbegyndes når type attributten enten er køb hvor der laves en drink, personale hvor maskinen åbnes op af en tekniker og lave kaffe gratis, men stadig bruger ingredienser. Eller spild som kunne være en test cyklus hvor maskinen sættes til at lave en drink for at se om den virker ordentligt. Desuden skal DrinkID være NOT NULL fordi systemet regner i hele drikke og det er nødvendigt for systemet at tjekke opskriften for en spildt espresso så den ved hvad der skal trækkes fra lageret. Dette sker ved JOIN nøgleord, der

forbinder opskrift og ingrediensLarger tabellerne hvor den brugte mængde fratrækkes den oprindelige mængde. Alt dette opdaterer så ingredienslageret. Hvor WHERE-funktionen sørger for at det kun sker med ingredienserne der er i drinken. Dernæst gøres der klar til at opdatere larger historikken ved INSERT INTO og SELECT bestemmer hvilke værdier der skal ind. Disse værdier findes ved at bruge FROM klausul og Where bruges til at opdatere DrinkID igen fordi systemet ikke kan tale på tværs af formål. Og kassen opdateres på tilsvarende måde.

```
-- querie 1 om at udlæse transaktioner
Select t.Tidspunkt,t.Type,t.metode,d.navn AS Drik, m.Navn AS Medarbejdere
FROM Transaktioner t
LEFT JOIN Drikkelse d ON t.Drikkelse_DrinkID=d.DrinkID
LEFT JOIN Medarbejdere m ON t.Medarbejdere_MedarbejderID = m.MedarbejderID
WHERE t.tidspunkt>='2022-05-01 00:00:00' And t.type='Køb';
-- querie 2 om at vise lagerindholdsstatus
SELECT Navn,Mængde,Enhed FROM IngrediensLager;
SELECT Beløb AS 'kontanter' FROM kasseoperat;
-- querie 3 om at rengøringshistorik
SELECT r.TID AS Dato_og_Tid, m.Navn AS Udført
FROM Rengøringshistorik r
JOIN Medarbejdere m ON r.Medarbejdere_MedarbejderID=m.MedarbejderID;
```

Den første querie udtager attributter gennem SELECT-klausul fra transaktionstabellen. Gennem AS alias går de under enklere navne som Drik og Medarbejdere. LEFT JOIN følger foreign-key og henter informationerne om type af drikke og hvilken medarbejder der er tale om. Querie 2 viser largerindhold gennem SELECT-klausul også pengene i kassen på den samme måde. Querie 3 henter rengøringshistorik og joiner tabellen til medarbejder tabellen så den pågældende medarbejder også står skrevet